

Door Gasket Project - Execution Checklist v1.0

Version: 1.0

Date: 2026-07-05

1) Core Objective

- After photo match or manual brand+model input, return a usable quote page quickly.
- When fields are incomplete, show in-loading status and trigger background enrichment tasks.
- Display matching data from DB only, with traceable source and confidence.
- Customer confirms before payment, then route to payment and keep a full order record.

2) Database Design

- Keep two core tables: refrigerator_products and product_gasket_specs (door-level gasket records).
- Define: brand, model, door_count, door_layout, manufacture_status, image_url, source_summary, data_confidence.
- Build brand/model alias tables for matched search behavior (e.g., True vs True Manufacturing).
- Build gasket section library: darts / push-in / screw-in / snap-in classes and geometry fields.
- Introduce relation: one gasket can match multiple products with clear source links and supplier part numbers.

3) Data Enrichment Flow

- AI extraction writes structured JSON first; app validates before DB write.
- Each missing field triggers independent async tasks: image, door layout, gasket dimensions/profile.
- No single slow task can block others; each field has timeout and retry count.
- Do not overwrite verified data with low-confidence records.
- Status levels: system_candidate, customer_confirmed, staff_verified, installed_verified.

4) Frontend Behavior

- Step 1: upload nameplate and read fields.
- Step 2: confirm brand/model and show current matching result status.
- Step 3: customer confirms or corrects dimensions/profile and proceeds to checkout.
- Never show 'not found'; use 'loading and matching now'.
- If no product image, show loading placeholder and continue normal queue for retrieval.

5) Admin and Security

- Admin pages are login-protected and Chinese UI where needed.
- Dashboard shows task queue health, hourly growth, and failed records.
- Purge unused crawlers and inactive scripts before release.

- Keep a rollback plan; avoid changing all data when one path fails.

6) Deployment and Monitoring

- Use one main deployment for public pages and background workers.
- Keep API costs under control; prefer non-paid enrichment paths when possible.
- Daily report: model count, image coverage, gasket coverage, queue backlog.
- Weekly review of completion quality and update the source rules.

7) Milestones

- M0: Data model lock + matching endpoint + protected admin.
- M1: Basic matching + async enrichment in production.
- M2: Checkout flow + shipping confirmation + order confirmation email.
- M3: Data quality + false-positive controls + long-term crawler optimization.

End of checklist.